

EXERCISE 03 - CASE Statements

DT MBENGA

DATA ANALYTICS

BRIGHTLEAN

10/09/2026

Q1

SQL Editor

Exercise 3 - Question 1

+

Run all (1000)

✓ 1 minute ago (21s)

workspace.default

Last edit was now

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

```
DROP TABLE IF EXISTS products;
CREATE TABLE products (
    product_id INT,
    product_name STRING,
    price DECIMAL(10,2)
);
INSERT INTO products VALUES
(1, 'Laptop', 1200.00),
(2, 'Phone', 800.00),
(3, 'Keyboard', 45.00),
(4, 'Monitor', 300.00),
(5, 'Mouse', 25.00);
/* Question 1 */
SELECT
    product_name,
    price,
    CASE
        WHEN price > 1000 THEN 'Expensive'
        WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
        WHEN price < 100 THEN 'Budget'
    END AS price_category
FROM products;
```

Add parameter

←

Results 4 of 4

→

Table

+

	product_name	price	price_category
1	Laptop	1200.00	Expensive
2	Phone	800.00	Mid-range
3	Keyboard	45.00	Budget
4	Monitor	300.00	Mid-range
5	Mouse	25.00	Budget

Q2

SQL Editor

Exercise 3 - Question 1Exercise 3- Question 2Exercise 3- Question 3

Run all (1000) 3 minutes ago (6s) workspace.default Last edit was now

```
5      customer_name STRING,
6      amount DECIMAL(10,2)
7  );
8  INSERT INTO orders VALUES
9  (1, 'Alice', 150.00),
10 (2, 'Bob', 560.00),
11 (3, 'Charlie', 999.99),
12 (4, 'Diana', 45.50),
13 (5, 'Ethan', 1200.00);
14
15 /* Q2: Classify orders based on amount */
16
17 SELECT
18     customer_name,
19     amount,
20     CASE
21         WHEN amount >= 1000 THEN 'High Value'
22         WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium Value'
23         WHEN amount < 500 THEN 'Low Value'
24     END AS order_value_category
25 FROM orders;
26
```

Add parameter

Results 5 of 5

Table

+

	customer_name	amount	order_value_category
1	Alice	150.00	Low Value
2	Bob	560.00	Medium Value
3	Charlie	999.99	Medium Value
4	Diana	45.50	Low Value
5	Ethan	1200.00	High Value

Q3

SQL Editor

Exercise 3 - Question 1Exercise 3- Question 2Exercise 3 - Ques

Run all (1000) Just now (5s) workspace.default Last edit was now

5emp_name STRING,
6department STRING,
7salary INT
8);
9
10INSERT INTO employees VALUES
11(1, 'John', 'IT', 85000),
12(2, 'Sara', 'HR', 60000),
13(3, 'Mark', 'IT', 75000),
14(4, 'Lucy', 'Finance', 95000),
15(5, 'Tom', 'HR', 55000);
16/* Q3: Classify employees based on department and salary */
17SELECT
18emp_name,
19department,
20salary,
21CASE
22WHEN department = 'IT' AND salary > 80000 THEN 'Senior IT'
23WHEN department = 'HR' AND salary > 55000 THEN 'Experienced HR'
24ELSE 'Staff'
25END AS position_level
26FROM employees;

Add parameter

← Results 4 of 4 →

Table +

	^A _C emp_name	^A _C department	¹ ₃ salary	^A _C position_level
1	John	IT	85000	Senior IT
2	Sara	HR	60000	Experienced HR
3	Mark	IT	75000	Staff
4	Lucy	Finance	95000	Staff
5	Tom	HR	55000	Staff

Q4

SQL Editor

Exercise 3 - Question 1

Exercise 3- Question 2

Run all (1000)

Just now (5s)

workspace

default

Last edit

```
5      score INT
6  );
7  INSERT INTO students VALUES
8  (1, 'Anna', 92),
9  (2, 'Ben', 76),
10 (3, 'Cara', 59),
11 (4, 'David', 83),
12 (5, 'Ella', 68);
13
14 /* Q4: Assign grades based on student scores */
15 SELECT
16     student_name,
17     score,
18     CASE
19         WHEN score >= 90 THEN 'A'
20         WHEN score BETWEEN 80 AND 89 THEN 'B'
21         WHEN score BETWEEN 70 AND 79 THEN 'C'
22         WHEN score BETWEEN 60 AND 69 THEN 'D'
23         ELSE 'F'
24     END AS grade
25 FROM students;
```

Add parameter

Results 4 of 4

Table

+

	^A _C student_name	¹ ₃ score	^A _C grade
1	Anna	92	A
2	Ben	76	C
3	Cara	59	F
4	David	83	B
5	Ella	68	D

Q5

SQL Editor

Exercise 3 - Question 1Exercise 3- Question 2Exercise 3 -

Run all (1000)Just now (4s)workspace.defaultLast edit was now

```
1 DROP TABLE IF EXISTS deliveries;
2 CREATE TABLE deliveries (
3     delivery_id INT,
4     delivery_time_minutes INT
5 );
6 INSERT INTO deliveries VALUES
7 (1, 45),
8 (2, 80),
9 (3, 30),
10 (4, 65),
11 (5, 100);
12 /* Q5: Classify delivery performance based on delivery time */
13 SELECT
14     delivery_id,
15     delivery_time_minutes,
16     CASE
17         WHEN delivery_time_minutes <= 30 THEN 'Fast'
18         WHEN delivery_time_minutes BETWEEN 31 AND 60 THEN 'On Time'
19         ELSE 'Late'
20     END AS performance
21 FROM deliveries;
```

Add parameter

Results 4 of 4Table+

	^{1 2 3} delivery_id	^{1 2 3} delivery_time_minutes	^{A B C} performance
1	1	45	On Time
2	2	80	Late
3	3	30	Fast
4	4	65	Late
5	5	100	Late

Q6

SQL Editor

Exercise 3 - Question 1

Exercise 3- Question 2

Exercise

Run all (1000)

Just now (4s)

workspace.default

Last edit was now

```
1 DROP TABLE IF EXISTS tickets;
2 CREATE TABLE tickets (
3     ticket_id INT,
4     issue_type STRING,
5     priority INT
6 );
7 INSERT INTO tickets VALUES
8 (1, 'Login issue', 1),
9 (2, 'Server down', 3),
10 (3, 'Slow system', 2),
11 (4, 'Email error', 2),
12 (5, 'Password reset', 1);
13 /* Q6: Convert numeric priority into a readable priority label */
14 SELECT
15     issue_type,
16     priority,
17     CASE
18         WHEN priority = 3 THEN 'High'
19         WHEN priority = 2 THEN 'Medium'
20         WHEN priority = 1 THEN 'Low'
21     END AS priority_label
22 FROM tickets;
```

Add parameter

Results 4 of 4

Table

+

	issue_type	priority	priority_label
1	Login issue	1	Low
2	Server down	3	High
3	Slow system	2	Medium
4	Email error	2	Medium
5	Password reset	1	Low

Q7

SQL Editor

Exercise 3 - Question 3Exercise 3 - Question 4Exercise 3 - Question 5

Run all (1000)Just now (4s)workspace.defaultLast edit was now

```
2 CREATE TABLE attendance (  
3     student_id INT,  
4     days_present INT,  
5     total_days INT  
6 );  
7 INSERT INTO attendance VALUES  
8 (1, 45, 50),  
9 (2, 30, 50),  
10 (3, 48, 50),  
11 (4, 25, 50),  
12 (5, 50, 50);  
13 /* Q7: Calculate attendance percentage and classify attendance */  
14  
15 SELECT  
16     student_id,  
17     (days_present * 100.0 / total_days) AS attendance_percentage,  
18     CASE  
19         WHEN (days_present * 100.0 / total_days) >= 90 THEN 'Excellent'  
20         WHEN (days_present * 100.0 / total_days) >= 75 THEN 'Good'  
21         ELSE 'Needs Improvement'  
22     END AS attendance_status  
23 FROM attendance;
```

Add parameter

Results 4 of 4

Table

	¹ ₃ student_id	.00 attendance_percentage	^A _C attendance_status
1	1	90.00000000000000	Excellent
2	2	60.00000000000000	Needs Improvement
3	3	96.00000000000000	Excellent
4	4	50.00000000000000	Needs Improvement
5	5	100.00000000000000	Excellent

Q8

SQL Editor

Exercise 3 - Question 3Exercise 3 - Question 4Exercise 3

Run all (1000) ✓ 1 minute ago (5s) workspace default Last edit was 1 minute

1 DROP TABLE IF EXISTS products_inventory;
2 CREATE TABLE products_inventory (
3 product_id INT,
4 stock_qty INT
5);
6 INSERT INTO products_inventory VALUES
7 (1, 5),
8 (2, 0),
9 (3, 25),
10 (4, 10),
11 (5, 3);
12 /* Q8: Classify products based on stock quantity */
13 SELECT
14 product_id,
15 stock_qty,
16 CASE
17 WHEN stock_qty = 0 THEN 'Out of Stock'
18 WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low Stock'
19 ELSE 'In Stock'
20 END AS stock_status
21 FROM products_inventory;

Add parameter

← Results 4 of 4 →

Table +

	^{1 2 3} product_id	^{1 2 3} stock_qty	^{A B C} stock_status
1	1	5	Low Stock
2	2	0	Out of Stock
3	3	25	In Stock
4	4	10	In Stock
5	5	3	Low Stock

Q9

SQL Editor

Exercise 3 - Question 3Exercise 3 - Question 4Exercise 3 - Q

Run all (1000)

Just now (4s)

workspace

default

Last edit was now

1

DROP TABLE IF EXISTS classes;

2

CREATE TABLE classes (

3

class_id INT,

4

subject STRING,

5

enrolled_students INT

6

);

7

INSERT INTO classes VALUES

8

(1, 'Math', 30),

9

(2, 'English', 25),

10

(3, 'Science', 15),

11

(4, 'Art', 5),

12

(5, 'History', 20);

13

/* Q9: Classify classes based on number of enrolled students */

14

SELECT

15

subject,

16

enrolled_students,

17

CASE

18

WHEN enrolled_students >= 25 THEN 'Large'

19

WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'

20

ELSE 'Small'

21

END AS class_size_category

22

FROM classes;

Add parameter

← Results 4 of 4 →

Table

+

	subject	enrolled_students	class_size_category
1	Math	30	Large
2	English	25	Large
3	Science	15	Medium
4	Art	5	Small
5	History	20	Medium

Q10

SQL Editor

Exercise 3 - Question 9

Exercise 3 - Question 10

Run all (1000)

Just now (5s)

workspace.

default

Last edit was now

```
1 DROP TABLE IF EXISTS payments;
2 CREATE TABLE payments (
3     payment_id INT,
4     amount DECIMAL(10,2),
5     payment_method STRING
6 );
7 INSERT INTO payments VALUES
8 (1, 50.00, 'Card'),
9 (2, 200.00, 'Cash'),
10 (3, 150.00, 'Card'),
11 (4, 75.00, 'PayPal'),
12 (5, 300.00, 'Cash');
13 /* Q10: Determine discount eligibility based on payment method and amount */
14 SELECT
15     payment_id,
16     payment_method,
17     amount,
18     CASE
19         WHEN payment_method = 'Cash' AND amount >= 200
20             THEN 'Eligible for Discount'
21         ELSE 'Not Eligible'
22     END AS discount_eligibility
23 FROM payments;
```

Add parameter

Results 4 of 4

Table

+

	payment_id	payment_method	amount	discount_eligibility
1	1	Card	50.00	Not Eligible
2	2	Cash	200.00	Eligible for Discount
3	3	Card	150.00	Not Eligible
4	4	PayPal	75.00	Not Eligible
5	5	Cash	300.00	Eligible for Discount

